

# Predicting Global Video Game Sales

Chris Gandiya

## Introduction

This project uses the **Video Game Sales with Ratings** dataset from Kaggle, which combines video game release information, sales data, and review scores. Each row represents a video game, with variables such as genre, platform, publisher, year of release, critic score, user score, and global sales. The goal of this analysis is to predict a game's global sales using non-sales features such as release year, review scores, genre, platform, and publisher group.

```
import os
import pickle

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV, RepeatedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import root_mean_squared_error, r2_score
```

## Data Cleaning and Feature Engineering

The dataset includes several sales variables, release details, and review information. I intentionally excluded the regional sales variables because global sales are calculated from regional sales.

```
gamesales = pd.read_csv("gamesales.csv")

games = gamesales[
    [
        "Global_Sales",
        "Genre",
        "Platform",
        "Publisher",
        "Year_of_Release",
        "Critic_Score",
        "User_Score"
    ]
]
```

```

].copy()

games = games.replace("", np.nan)
games["User_Score"] = games["User_Score"].replace("tbd", np.nan)

games["Global_Sales"] = pd.to_numeric(games["Global_Sales"], errors="coerce")
games["Year_of_Release"] = pd.to_numeric(games["Year_of_Release"],
errors="coerce")
games["Critic_Score"] = pd.to_numeric(games["Critic_Score"], errors="coerce")
games["User_Score"] = pd.to_numeric(games["User_Score"], errors="coerce")

games = games.dropna(subset=["Global_Sales"])

top_publishers = games["Publisher"].value_counts().head(10).index

games["Publisher_Grouped"] = np.where(
    games["Publisher"].isin(top_publishers),
    games["Publisher"],
    "Other"
)

games = games.drop(columns=["Publisher"])

print(f"Rows: {games.shape[0]}")
print(f"Columns: {games.shape[1]}")

Rows: 16719
Columns: 7

```

Blank values and "tbd" user scores were converted to missing values so they could be handled consistently. I also grouped publishers outside the top 10 into an "Other" category. This keeps the feature space smaller and makes the publisher variable easier to interpret.

```

summary_table = pd.DataFrame({
    "Variable": games.columns,
    "Missing Values": games.isna().sum().values,
    "Missing Rate": games.isna().mean().values
})

summary_table

```

|   | Variable     | Missing Values | Missing Rate |
|---|--------------|----------------|--------------|
| 0 | Global_Sales | 0              | 0.000000     |
| 1 | Genre        | 2              | 0.000120     |

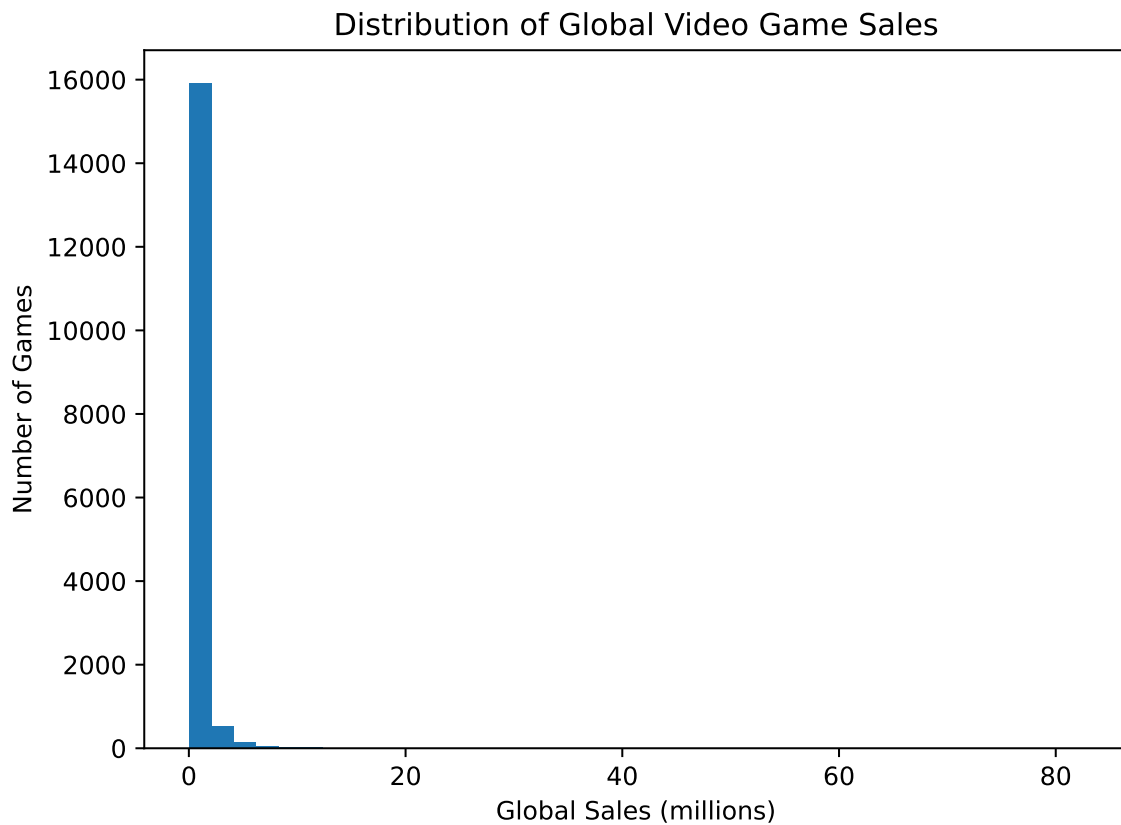
|   | Variable          | Missing Values | Missing Rate |
|---|-------------------|----------------|--------------|
| 2 | Platform          | 0              | 0.000000     |
| 3 | Year_of_Release   | 269            | 0.016089     |
| 4 | Critic_Score      | 8582           | 0.513308     |
| 5 | User_Score        | 9129           | 0.546025     |
| 6 | Publisher_Grouped | 0              | 0.000000     |

The missing-value summary shows that review scores are missing for many games. Instead of dropping every game with a missing predictor, I keep those rows and handle missing values inside the preprocessing pipeline.

## Exploratory Data Analysis

I started by examining the response variable. Before choosing a model, it is important to understand whether or not global sales are roughly symmetric.

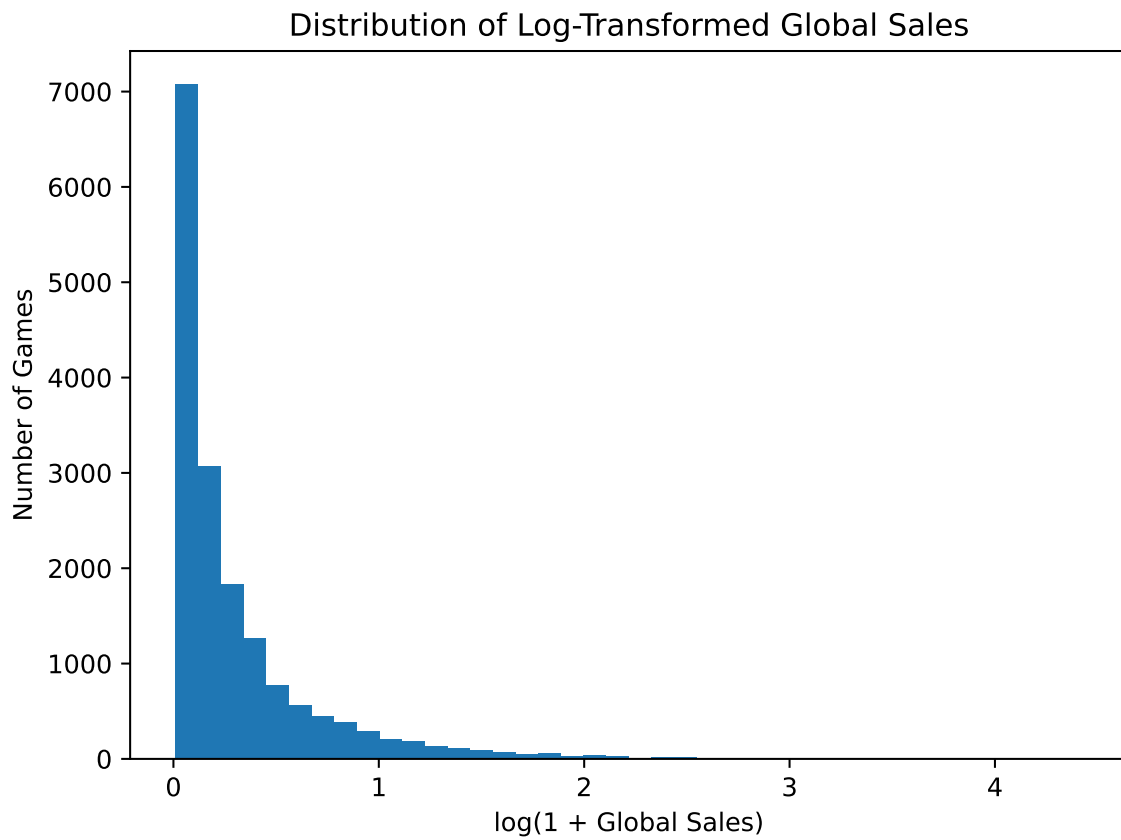
```
plt.figure(figsize=(7, 5))
plt.hist(games["Global_Sales"], bins=40)
plt.xlabel("Global Sales (millions)")
plt.ylabel("Number of Games")
plt.title("Distribution of Global Video Game Sales")
plt.show()
```



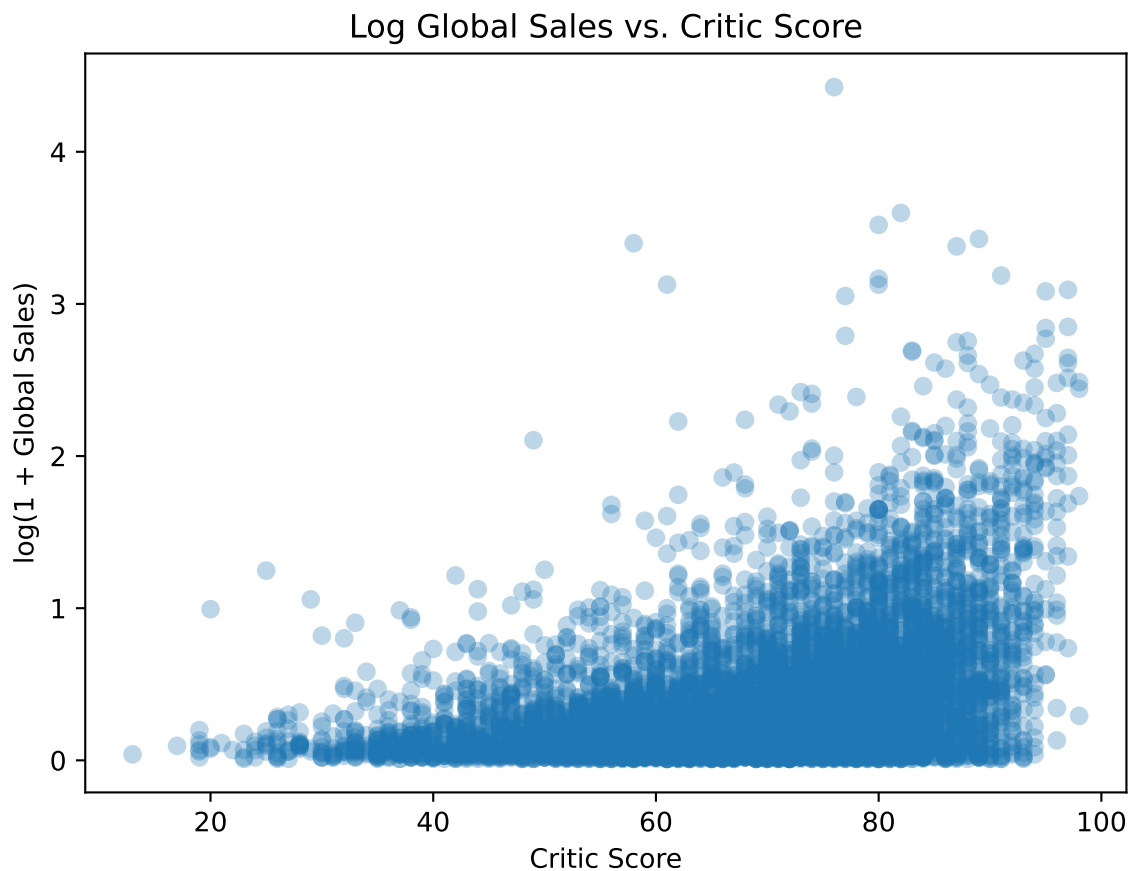
Global sales are heavily right-skewed. Most games sell relatively few copies, while a small number of blockbuster games sell far more than the rest of the dataset. Because of this, I created a log-transformed version of global sales for modeling.

```
games["Log_Global_Sales"] = np.log1p(games["Global_Sales"])

plt.figure(figsize=(7, 5))
plt.hist(games["Log_Global_Sales"], bins=40)
plt.xlabel("log(1 + Global Sales)")
plt.ylabel("Number of Games")
plt.title("Distribution of Log-Transformed Global Sales")
plt.show()
```

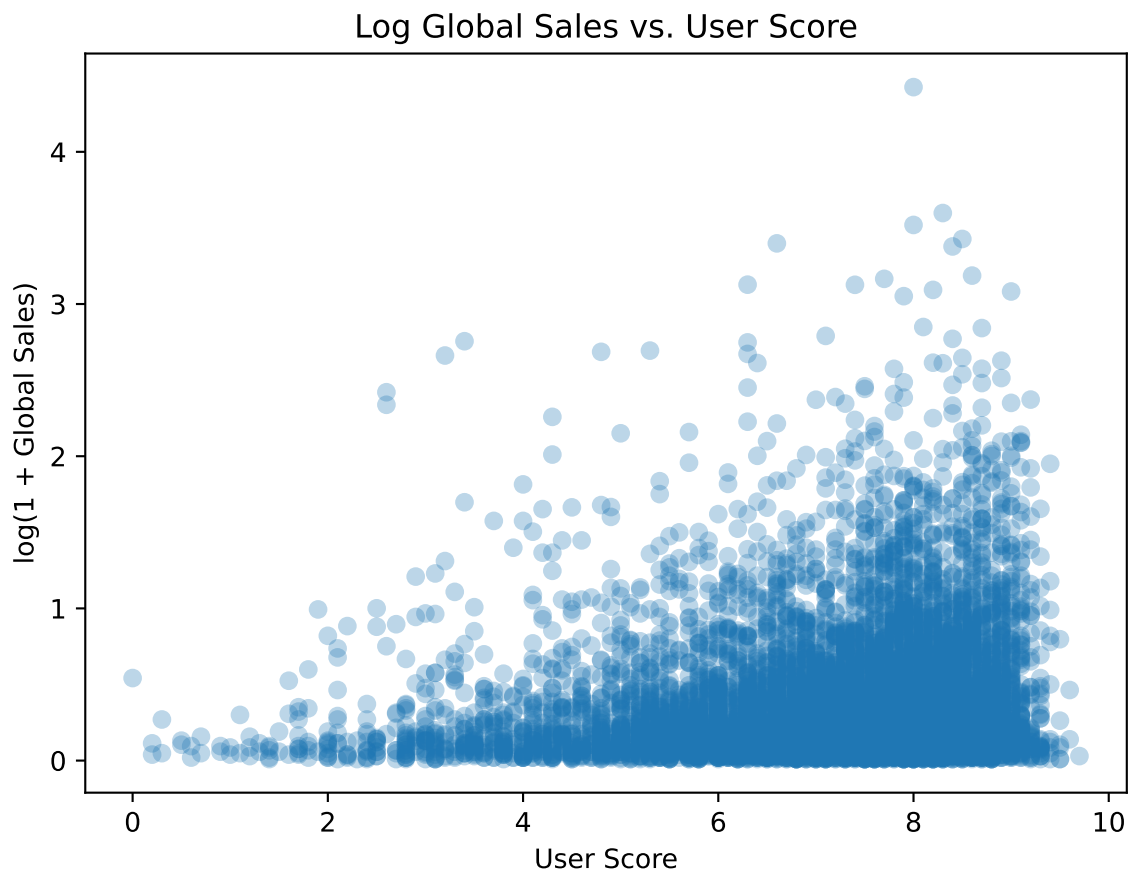


```
plt.figure(figsize=(7, 5))
plt.scatter(games["Critic_Score"], games["Log_Global_Sales"], alpha=0.3)
plt.xlabel("Critic Score")
plt.ylabel("log(1 + Global Sales)")
plt.title("Log Global Sales vs. Critic Score")
plt.show()
```



Games with higher critic scores tend to have somewhat higher log sales, but the pattern is not perfect. There are many well-reviewed games with modest sales, which suggests that review quality helps but does not come close to explaining sales by itself.

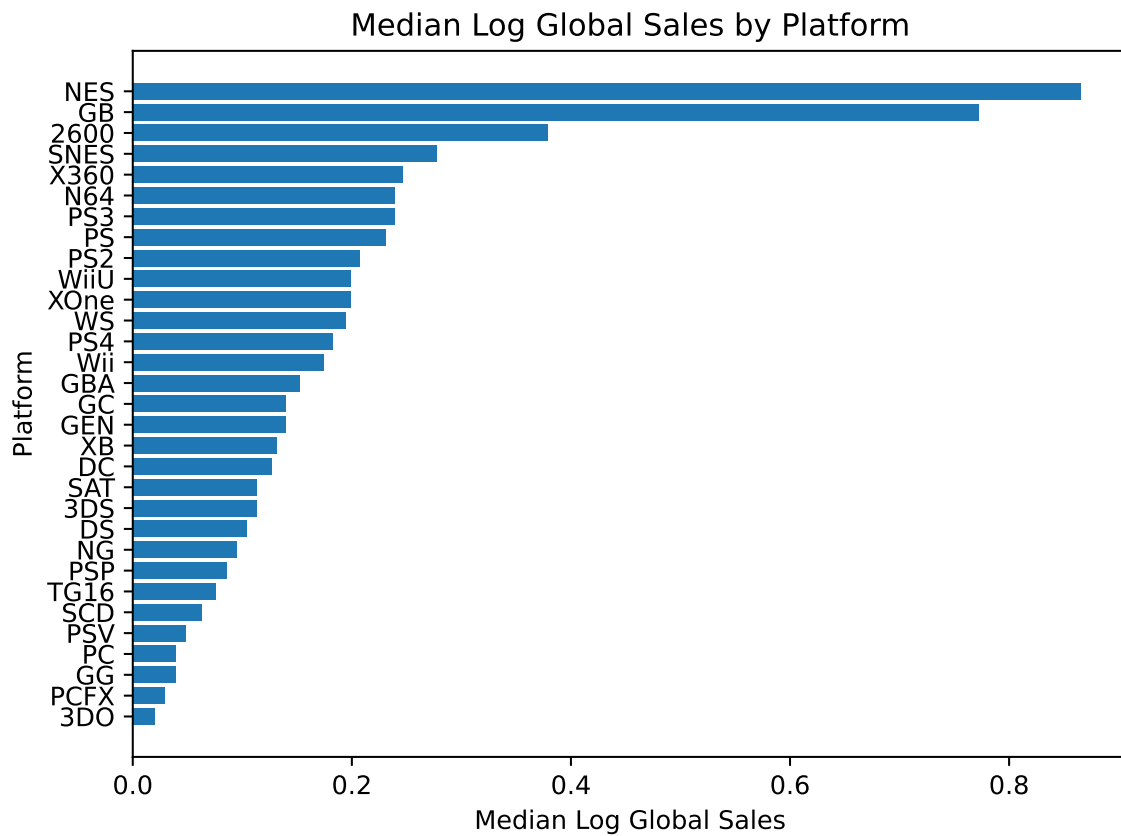
```
plt.figure(figsize=(7, 5))
plt.scatter(games["User_Score"], games["Log_Global_Sales"], alpha=0.3)
plt.xlabel("User Score")
plt.ylabel("log(1 + Global Sales)")
plt.title("Log Global Sales vs. User Score")
plt.show()
```



User score appears even noisier than critic score. Some highly rated games sold very little, and some lower-rated games still sold relatively well. This makes user score worth including, but I would not expect it to be one of the strongest predictors on its own.

```
platform_sales = games.groupby("Platform")["Log_Global_Sales"].median().sort_values()

plt.figure(figsize=(7, 5))
plt.barh(platform_sales.index, platform_sales.values)
plt.xlabel("Median Log Global Sales")
plt.ylabel("Platform")
plt.title("Median Log Global Sales by Platform")
plt.show()
```

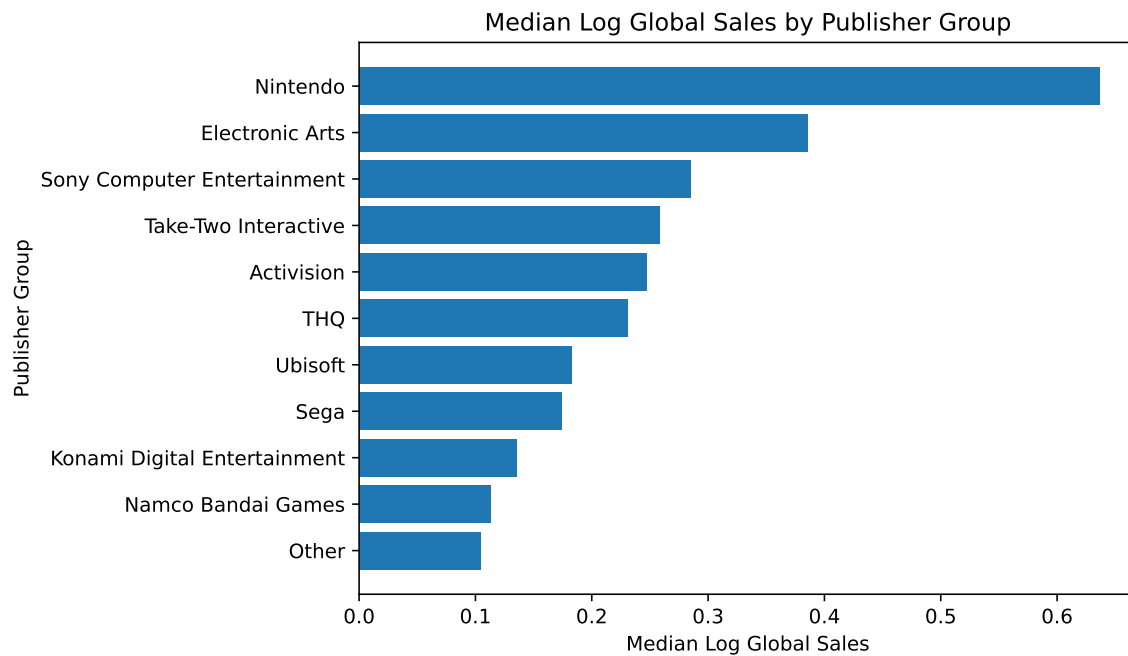


Median log sales differ significantly across platforms, so platform likely contains useful predictive information.

```
publisher_sales =
games.groupby("Publisher_Grouped")["Log_Global_Sales"].median().sort_values()

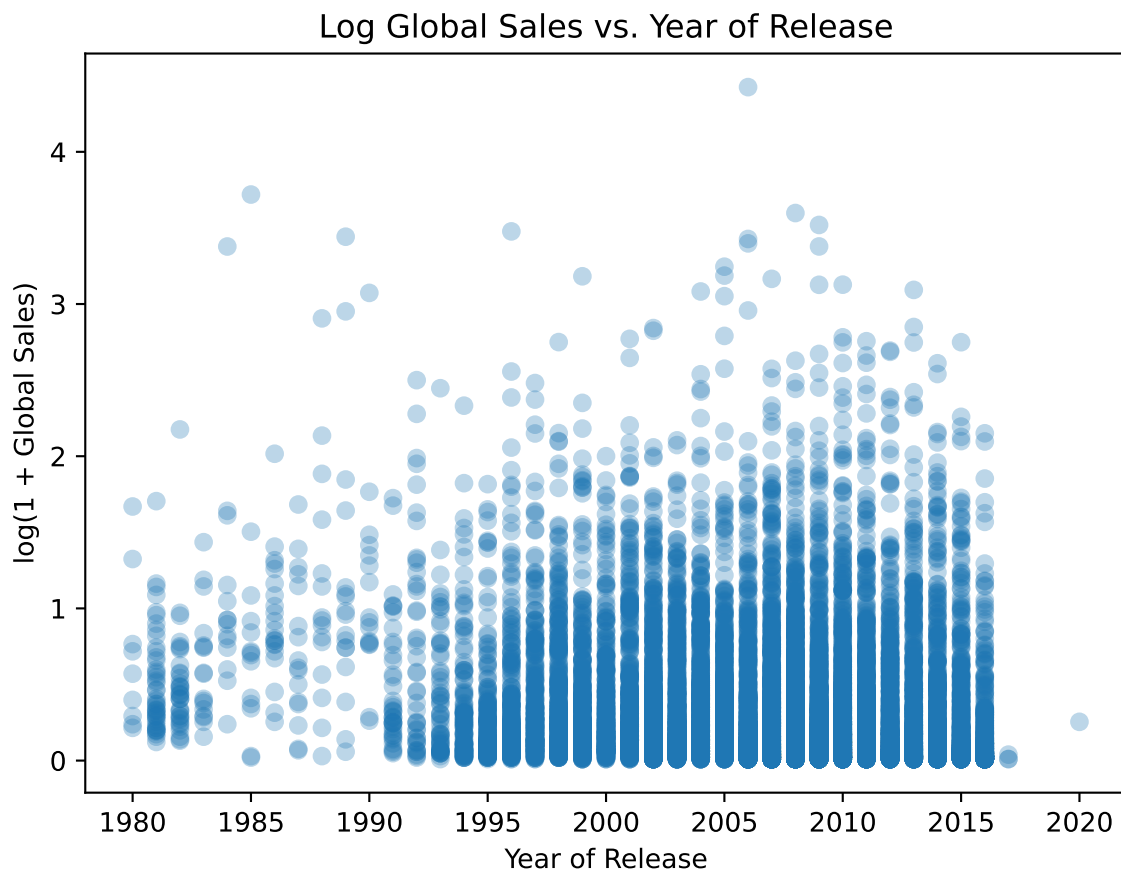
plt.figure(figsize=(7, 5))
plt.barh(publisher_sales.index, publisher_sales.values)
plt.xlabel("Median Log Global Sales")
plt.ylabel("Publisher Group")
plt.title("Median Log Global Sales by Publisher Group")
plt.show()
```





Median log sales also vary by publisher group. Larger publishers may benefit from stronger marketing, distribution, brand recognition, or established franchises. Since smaller publishers were grouped into "Other", this feature is useful for prediction but should not be treated as a detailed comparison of every publisher.

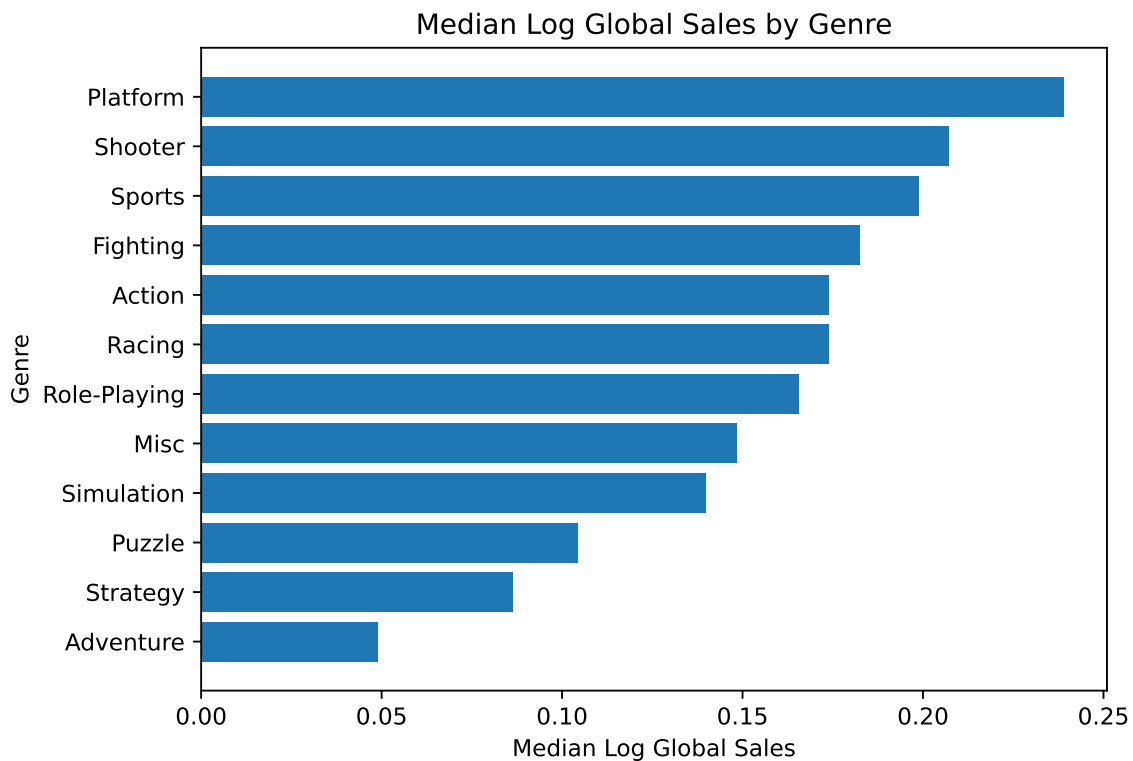
```
plt.figure(figsize=(7, 5))
plt.scatter(games["Year_of_Release"], games["Log_Global_Sales"], alpha=0.3)
plt.xlabel("Year of Release")
plt.ylabel("log(1 + Global Sales)")
plt.title("Log Global Sales vs. Year of Release")
plt.show()
```



Log global sales vary across release years, but the relationship is not strongly linear. Year may still be useful because it captures changes in the gaming market over time.

```
genre_sales = games.groupby("Genre")["Log_Global_Sales"].median().sort_values()

plt.figure(figsize=(7, 5))
plt.barh(genre_sales.index, genre_sales.values)
plt.xlabel("Median Log Global Sales")
plt.ylabel("Genre")
plt.title("Median Log Global Sales by Genre")
plt.show()
```



Median log sales differ across genres, which suggests genre may contain useful predictive information. However, genre is not enough to explain sales on its own, since individual games within each genre can still vary a lot.

## Modeling Approach

I split the data into training and test sets before fitting any models. The training set is used for fitting and cross-validation, while the test set is saved for the final performance comparison. This gives a better estimate of how the models perform on predicting sales for games they have not already seen.

```
X = games.drop(columns=["Global_Sales", "Log_Global_Sales"])
y = games["Log_Global_Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=4025
)

num_cols = ["Year_of_Release", "Critic_Score", "User_Score"]
cat_cols = ["Genre", "Platform", "Publisher_Grouped"]
```

```

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols)
])

```

The preprocessing pipeline handles missing values and categorical variables at the same time. Numeric predictors are imputed with the median, while categorical predictors are imputed with the most common category and then one-hot encoded.

### Baseline Model: Linear Regression

I first fit a linear regression model as a baseline. This gives a simple point of comparison before using a more flexible model.

```

linreg_pipe = Pipeline([
    ("prep", preprocessor),
    ("model", LinearRegression())
])

linreg_pipe.fit(X_train, y_train)

linreg_pred = linreg_pipe.predict(X_test)

linreg_rmse = root_mean_squared_error(y_test, linreg_pred)
linreg_r2 = r2_score(y_test, linreg_pred)

linreg_rmse, linreg_r2
(0.3313982003174651, 0.2594913137467153)

```

### Random Forest Model

I then fit a random forest model. This model is useful here because sales may depend on interactions that a basic linear model does not capture. For example, a genre may perform differently across platforms, or review scores may matter more for some types of games than others.

```

rf_pipe = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=100,

```

```

        random_state=4025,
        n_jobs=-1
    ))
])

rf_pipe.fit(X_train, y_train)

rf_pred = rf_pipe.predict(X_test)

rf_rmse = root_mean_squared_error(y_test, rf_pred)
rf_r2 = r2_score(y_test, rf_pred)

rf_rmse, rf_r2

(0.30664789597578906, 0.3659699463916347)

```

### Model Tuning

To check whether the random forest could be improved, I used repeated cross-validation to tune `max_features`. This parameter controls how many predictors are considered at each split. Smaller values can make the trees more different from each other, while larger values allow each split to consider more information.

```

rf_tune_pipe = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=100,
        random_state=4025,
        n_jobs=-1
    ))
])

cv = RepeatedKFold(
    n_splits=5,
    n_repeats=3,
    random_state=4025
)

param_grid = {
    "model__max_features": ["sqrt", "log2", 0.5, None]
}

cache_file = "rf_sales_grid.pkl"

if os.path.exists(cache_file):
    with open(cache_file, "rb") as f:

```

```

        rf_grid = pickle.load(f)
    else:
        rf_grid = GridSearchCV(
            rf_tune_pipe,
            param_grid=param_grid,
            cv=cv,
            scoring="neg_root_mean_squared_error",
            n_jobs=-1
        )

        rf_grid.fit(X_train, y_train)

        with open(cache_file, "wb") as f:
            pickle.dump(rf_grid, f)

    print("Best parameters:", rf_grid.best_params_)
    print("Best CV RMSE:", -rf_grid.best_score_)

    Best parameters: {'model__max_features': 'sqrt'}
    Best CV RMSE: 0.30788118265283754

    best_rf = rf_grid.best_estimator_

    best_rf_pred = best_rf.predict(X_test)

    best_rf_rmse = root_mean_squared_error(y_test, best_rf_pred)
    best_rf_r2 = r2_score(y_test, best_rf_pred)

    results = pd.DataFrame({
        "Model": ["Linear Regression", "Random Forest", "Tuned Random Forest"],
        "RMSE": [linreg_rmse, rf_rmse, best_rf_rmse],
        "R2": [linreg_r2, rf_r2, best_rf_r2]
    })

    results.round(3)

```

|   | Model               | RMSE  | R2    |
|---|---------------------|-------|-------|
| 0 | Linear Regression   | 0.331 | 0.259 |
| 1 | Random Forest       | 0.307 | 0.366 |
| 2 | Tuned Random Forest | 0.305 | 0.371 |

The random forest models outperformed the linear regression baseline. Linear regression had an RMSE of 0.331 and an ( $R^2$ ) of 0.259. The untuned random forest improved performance, with an RMSE of 0.307 and an ( $R^2$ ) of 0.366. Because the response variable was log-transformed, RMSE is reported on the log-sales scale rather than directly in millions of copies.

The tuned random forest selected `max_features = "sqrt"`, with a cross-validated RMSE of about 0.307. On the test set, the tuned model had an RMSE of 0.305 and an ( $R^2$ ) of 0.374. This is nearly the same as the untuned random forest, so tuning did not meaningfully improve performance in this case.

Overall, the random forest is the strongest model, but the improvement from tuning is small. The best model explains about 37% of the variation in log global sales, which is useful but still leaves substantial unexplained variation. That makes sense because commercial success likely depends on important factors missing from this dataset, such as marketing budget, franchise popularity, release timing, digital sales, and competition from other releases.

## Model Interpretation

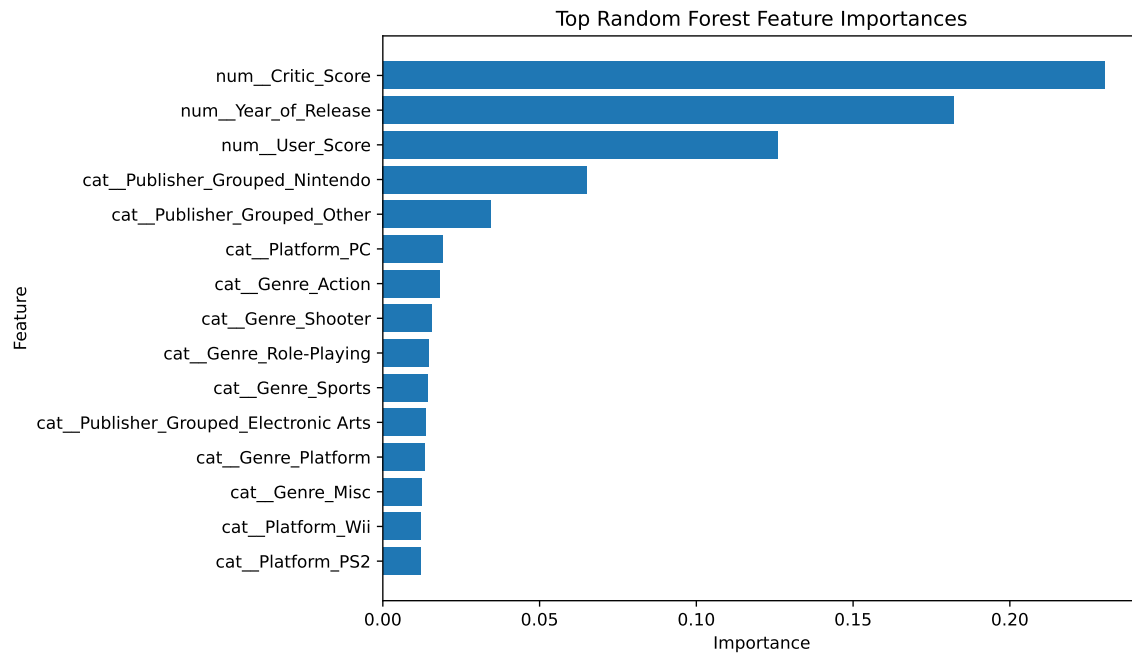
To better understand the random forest, I examined the variables with the highest feature importance values.

```
feature_names = best_rf.named_steps["prep"].get_feature_names_out()
importances = best_rf.named_steps["model"].feature_importances_

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values("Importance", ascending=False).head(15)

importance_df = importance_df.sort_values("Importance")

plt.figure(figsize=(8, 6))
plt.barh(importance_df["Feature"], importance_df["Importance"])
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top Random Forest Feature Importances")
plt.show()
```



The feature importance plot shows which predictors the random forest used most when predicting log global sales. These importances are useful for understanding the model, but they should not be interpreted causally. For example, if a platform or publisher group is important, that does not mean it directly caused higher sales. It may also reflect franchise popularity, marketing, console popularity, or other factors not included in the data.

## Limitations

There are several limitations to this analysis. First, review scores are missing for a large portion of the dataset. I used imputation so that these rows could still be included, but the values may not be missing at random. Games without critic or user scores may be different from games that were reviewed.

Second, this model uses review scores, which are usually only available after a game is released. Because of that, the project is better understood as a retrospective sales prediction analysis rather than a true pre-release forecasting model.

Third, the dataset does not include several major factors that likely affect sales, such as marketing budget, franchise history, release month, digital sales, or competition from other games. Finally, grouping smaller publishers into "Other" makes the model easier to interpret, but it also hides differences between smaller publishers.

## Conclusion

This analysis found that global video game sales are difficult to predict from release information and review scores alone. The random forest performed better than the linear regression model, which suggests that nonlinear patterns and interactions between variables like platform, genre, publisher group, and review scores are useful for prediction. However, tuning the random forest



did not meaningfully improve performance, so the simpler untuned forest would be a reasonable final model.

The best model explained about 37% of the variation in log global sales. That is useful, but it also shows that a lot of variation remains unexplained. This makes sense because the dataset does not include major business factors such as marketing budget, franchise popularity, digital sales, release timing, or competition from other games.

Overall, the results suggest that review scores and release features provide some signal, but they are not enough to fully explain commercial success. A stronger analysis would need additional business and market data, especially variables that capture audience awareness and publisher investment before release.